Lucera Trading Systems Challenge: Project Starter Guide

This guide offers starting points, not prescribed solutions. Teams may choose one of these ideas, adapt one, or pursue a different trading-technology problem.

For a 24-hour hackathon, choose one core claim and prove it well. Start with a thin working path, define what must always be true, add adversarial tests, and then measure behavior under a disclosed workload. Judges appreciate engineering intuition: sensible scope, explicit assumptions, simple abstractions, careful edge-case handling, and an honest understanding of tradeoffs.

1. Python tooling for Simple Binary Encoding (SBE)

The opportunity

Simple Binary Encoding is a high-performance binary message format used in low-latency financial systems. Its official reference repository currently lists implementations for Java, C, C++, C#, Go, and Rust, but not Python. A team could improve the Python ecosystem around SBE without attempting an entire implementation in one weekend.

A realistic 24-hour scope

Choose one narrow vertical slice:

- decode a useful subset of an SBE schema and message stream in Python;
- generate typed Python decoders from a small SBE schema;
- add support for one missing schema feature to an existing Python SBE project;
- provide a Python wrapper around an existing native codec;
- build inspection or debugging tools that render SBE messages as structured JSON; or
- contribute tests, documentation, compatibility fixes, or measurable performance improvements to an existing implementation.

Where to start

- Read the official SBE repository and its sample schemas.
- Review the history and design discussion in Python generator issue #665.
- Evaluate existing third-party work, such as python-systems/sbe-codegen, before deciding whether to extend, wrap, test, or replace something.
- Check the chosen repository's open issues, license, contribution guide, style, supported Python versions, and test commands before writing code.

Correctness ideas

- Encoding followed by decoding returns the same values for every supported type.

- Invalid lengths, truncated buffers, unknown templates, and unsupported schema features fail explicitly.
- Decoder bounds checks prevent reads past the supplied buffer.
- Known SBE fixtures produce results consistent with a reference implementation.
- Schema versioning and optional fields behave consistently for the supported subset.

Use fixed fixtures plus property-based or fuzz testing where practical. Clearly document unsupported features rather than silently decoding them incorrectly.

Performance evidence

Measure messages decoded per second, latency per message, allocations, or peak memory on a fixed fixture set. Compare against a simple baseline only when the comparison is fair. Correctness and API quality matter more than a misleading headline benchmark.

Open-source standard

The submission should be reviewable as if it were going upstream: focused commits or a clean patch, tests, documentation, formatting/lint compliance, compatibility notes, and a clear explanation of the change. A maintainer does not need to merge the work during HackRU.

2. Deterministic FIX explainer with bounded AI assistance

The opportunity

FIX messages are compact and context-heavy. A useful explainer could translate a raw message into plain language, link tags and enum values to authoritative definitions, identify missing context, and help a developer understand what happened.

The strongest design keeps protocol facts deterministic. An AI model may summarize, answer follow-up questions, or improve presentation, but it should not invent tag meanings, order state, or causality.

A realistic 24-hour scope

Build a tool that accepts one message or a short sequence and returns:

- parsed header, body, and trailer fields;
- field names and decoded enum values;
- message type and likely business purpose;
- validation errors or missing required context;
- a concise plain-language explanation with citations to the fields used; and
- an explicit distinction between known facts, inferred context, and unknowns.

A CLI or small web interface is enough. The protocol interpretation is the product; visual polish is secondary.

Where to start

- Use FIXimate as a human-readable protocol reference.

- Use the machine-readable FIX Orchestra repository for field, message, component, and code-set definitions.
- For Python, examine pyfixorchestra.
- Start with a small set of common order-flow messages, such as New Order Single, Execution Report, Order Cancel Request, and Order Cancel Reject.
- Create synthetic or sanitized fixtures. Do not send confidential production logs to an external model.

Suggested architecture

1. A deterministic parser tokenizes `tag=value` fields and preserves ordering and duplicates.
2. A specification layer resolves message types, field names, enums, required fields, and repeating groups.
3. A rules layer derives only facts supported by the messages.
4. An optional AI layer receives structured, redacted facts—not unrestricted raw logs—and produces readable explanations.
5. The output cites the tags or messages supporting each important statement.

Correctness ideas

- The same input and specification version produce the same structured interpretation.
- Unknown tags and malformed fields are preserved and reported, not dropped.
- Repeating groups and duplicate tags are handled explicitly.
- AI output cannot override deterministic validation results.
- Every factual claim in the explanation can be traced to input fields or protocol metadata.

Performance evidence

Measure time to parse and explain a fixed message set, peak memory, or the ratio of supported to unsupported fields. If an AI API is used, report deterministic parsing separately from model latency and avoid presenting model variability as parser performance.

3. Real-time FIX log analyzer and trading-activity report

The opportunity

Support engineers and trading teams often need to inspect many FIX logs, correlate messages across sessions or files, reconstruct order lifecycles, and explain what traded and why. Manual searching becomes slow and error-prone at scale.

Build a tool that consumes historical files, a live stream, or both and produces a deterministic report of observed trading activity.

A realistic 24-hour scope

Support one well-defined workflow:

- ingest multiple log files or follow an appended file;
- normalize timestamps and session identifiers;
- parse and validate FIX messages;

- correlate a small set of order identifiers;
- reconstruct new, replace, cancel, reject, and fill events;
- report quantities, prices, statuses, and unresolved gaps; and
- emit human-readable HTML/Markdown plus machine-readable JSON.

Do not try to support every FIX version, venue dialect, and asset class. State the supported message set and assumptions.

Where to start

- Generate sanitized fixtures for several interleaved orders and sessions.
- Begin with deterministic parsing and correlation before adding a UI or AI summary.
- Use the FIX Orchestra resources or FIXimate for protocol metadata.
- Consider a pipeline such as `reader -> parser -> normalizer -> correlator -> lifecycle reducer -> report`.
- Keep vendor-specific log prefixes separate from the FIX payload parser.

Report ideas

- session health: logon/logout, sequence gaps, rejects, and resend activity;
- order lifecycle: accepted, replaced, canceled, rejected, partially filled, or filled;
- executions: symbol, side, quantity, price, and timestamps;
- latency: selected request-to-acknowledgment or order-to-fill intervals;
- unresolved activity: orphan responses, missing requests, conflicting states, or incomplete logs; and
- evidence: source file, line, timestamp, sequence number, and identifiers behind each conclusion.

Correctness ideas

- Reprocessing identical logs produces identical structured reports.
- File chunking or ingestion order does not change results once events are ordered by the declared policy.
- Duplicate messages are handled according to a documented key and do not double-count fills.
- Quantities and state transitions satisfy documented invariants.
- Ambiguous or missing evidence is reported as unknown rather than guessed.

Performance evidence

Measure messages or bytes processed per second, time to first report, peak memory, or update latency while tailing a file. Use a generated workload with known expected totals so speed and correctness can be checked together.

Optional AI extension

Add AI only after the deterministic report works. Give the model the structured report and supporting references, ask it to summarize or answer questions, and require it to distinguish facts from hypotheses. Redact credentials, account data, counterparties, and other sensitive values.

4. API-driven backtesting engine

The opportunity

A backtester is most useful when data access, strategy logic, execution assumptions, accounting, and results are separated by clear APIs. This makes strategies repeatable and allows data or execution models to be replaced without rewriting the engine.

A realistic 24-hour scope

Build a vertical slice that:

- loads one asset class and one frequency through a data-provider interface;
- exposes a small strategy API;
- processes events in deterministic timestamp order;
- simulates market or limit orders using a documented fill model;
- tracks cash, positions, fees, and profit and loss; and
- emits trades, an equity curve, and a reproducible run manifest.

Focus on trustworthy mechanics rather than sophisticated strategy returns. A trivial moving-average or threshold strategy is enough to exercise the engine.

Where to start

- Use generated fixtures first so expected outcomes can be calculated by hand.
- Add an adapter for Alpaca historical market data, yfinance, CSV, or another permitted source.
- Define interfaces such as `DataProvider`, `Strategy`, `ExecutionModel`, `Portfolio`, and `ResultStore`.
- Cache or snapshot external data with its symbol, time range, interval, provider, timezone, and retrieval time so runs remain explainable.
- Keep data acquisition outside the core simulation loop.

Correctness ideas

- A strategy can only observe information available at the simulated time; no look-ahead is possible.
- Events with equal timestamps follow a documented deterministic ordering.
- Cash, positions, fees, and realized/unrealized P&L reconcile after every fill.
- Splits, missing bars, timezones, and market calendars are either handled or explicitly out of scope.
- Identical code, configuration, and data snapshot produce identical trades and results.

Performance evidence

Measure bars or events processed per second, memory versus dataset size, startup/data-loading cost, or runtime scaling across symbols. Report data-download time separately from simulation time.

Useful extensions

- pluggable commission and slippage models;
- parameter sweeps without accidental data leakage;
- event-level replay for debugging;
- checkpointing or distributed runs;
- comparison of strategy decisions under two execution models; or
- a REST or Python API that submits runs and retrieves immutable results.

How to shape another idea

Any trading-technology project can use this outline:

1. **User and problem:** who has the problem, and what decision or workflow improves?
2. **Core claim:** what one capability will work by the end of HackRU?
3. **Boundary:** what is explicitly unsupported?
4. **Correctness:** what must never be false?
5. **Evidence:** which fixtures, tests, or reference implementation verify it?
6. **Interfaces:** what must communicate, or what public library interface will users call?
7. **Measurement:** which constrained metric demonstrates useful behavior?
8. **Failure behavior:** what happens with invalid, duplicated, delayed, missing, or out-of-order input?
9. **Reproduction:** can a judge run the demo, tests, and measurement from the repository?
10. **Next limit:** what bottleneck or design change would come next?

Final advice

- Build the smallest honest version of the idea first.
- Prefer deterministic behavior for parsing, accounting, validation, and state transitions.
- Use AI where it improves access or explanation, not as a substitute for protocol logic.
- Keep external services behind interfaces and provide local fixtures for judging.
- Never commit credentials or confidential market, client, account, or counterparty data.
- Explain engineering decisions and rejected alternatives. Good judgment is part of the submission.